

```

        if reportsList is None:
            reportsList = []
        self.reportsList = reportsList
    ❸ super().__init__(name, title)

    ❹ def getReports(self):
        return self.reportsList

    ❺ def payPerYear(self, giveBonus=False):
        pay = self.salary
        if giveBonus:
            pay = pay + (.10 * self.salary) # add a bonus of 10%
        ❻ print(self.name, 'gets a bonus for good work')
        return pay

```

Listing 10-2: The `Manager` class, implemented as a subclass of the `Employee` class

In the `class` statement ❶, you can see that this class inherits from the `Employee` class because `Employee` is inside the parentheses after the name `Manager`.

The `__init__()` method of the `Employee` class expects a name, a title, and an optional rate per hour. A manager is a salaried employee and manages a number of employees, so the `__init__()` method of the `Manager` class expects a name, a title, a salary, and a list of employees. Adhering to the principle of coding by difference, the `__init__()` method starts by initializing anything the `__init__()` method of the `Employee` class doesn't do. Therefore, we save the salary and `reportsList` in similarly named instance variables ❷.

Next we want to call the `__init__()` method of the `Employee` base class ❸. Here, I am calling the built-in function `super()`, which asks Python to figure out which class is the base class (often referred to as the *superclass*) and call that class's `__init__()` method. It also adjusts the arguments to include `self` as the first argument in this call. Therefore, you can think of this line as translating to:

```
Employee.__init__(self, name, title)
```

In fact, coding that line this way would work perfectly well; using the call to `super()` is simply a much cleaner way to write the call without having